```
printQuote($quote);
print $quote;
```

```
Output:
We're bona fide, we're not from London
We've gone on holiday by mistake.
```

This should look familiar, if not see the section "Reference vs. Value" above. Here we can see that the global `$quote` is not affected when we overwrite the data passed in during the call to `printQuote()` (the first line of the function body). This is because lexical scope changes to local when `printQuote()` is invoked, and this includes any variables used as function arguments. We have two variables called `$quote` in this code: one in global scope and one scoped locally to `printQuote()`. They share a name, but they are protected from each other. We've already seen earlier in the chapter how code like this works. We know that values are passed by-value by default during function calls. The change in lexical scope is the mechanism that facilitates this.

If we had defined `$quote` as a reference in the declaration – `function printQuote (string &$quote): void { ... }` – then we would still have a locally scoped `$quote` variable, but its value would not be a copy of the global string. A locally scoped `&$quote` would be a reference to the global `$quote`, and any changes to the local `$quote` would propagate to global `$quote`.

Local Scope and Static Variables

Earlier in this section, we established that any variables created within local scope are destroyed when the scope changes back to global (when the boundary between local and scope is crossed). The following code demonstrates this behavior:

```
function getCount(): int
{
    if (!isset($counter)) {
        $counter = 0;
    }

    return $counter++;
}
```